

# Reviewer Pack — Schur-Weyl Decomposition Component Count vs. Monotone Circui...

Entry 965ac372534a · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-12 11:48:50 UTC

## Schur-Weyl Decomposition Component Count vs. Monotone Circuit Size for Permanent-like CNFs

Entry ID: 965ac372534a **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-12 11:48:50 UTC

### 1. Conjecture

**Field A** (mathematical branch): Schur-Weyl Duality **Field B** (complexity object): Monotone Circuit Size

**Statement:**

For any CNF  $\Phi$  with  $n$  variables and  $m$  clauses, let  $C(\Phi)$  denote the number of irreducible components in the Schur-Weyl decomposition of the tensor product of its clause indicator vectors. Then, for all unsatisfiable  $\Phi$  with  $m = \Theta(n^2)$ ,  $C(\Phi) \geq 2^{\lfloor n/2 \rfloor} / \text{poly}(n)$  and the monotone circuit size of  $\Phi$  is  $\Omega(C(\Phi) * \log n)$ .

**Rationale (proposer’s reasoning):**

Schur-Weyl duality decomposes tensor products into symmetric/antisymmetric subspaces, revealing structural complexity. Permanent-like CNFs exhibit richer decomposition patterns due to their symmetric nature, which may correlate with inherent monotone circuit lower bounds via representation-theoretic obstructions.

**Taxonomy category:** GCT\_DET\_PERM (status at proposal time: partially\_alive)

### 2. Pre-registration (Popper-style)

**Hash (SHA-256 prefix):** ff28269d5e6ae5e6

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

**Acceptance criterion:**

default:  $\geq 80\%$  seeds must support, no counterexample

### 3. Barrier filter (F1)

**Final:** PASS

| Barrier        | Verdict | Confidence | LLM <sub>1</sub> | LLM <sub>2</sub> |
|----------------|---------|------------|------------------|------------------|
| RELATIVIZATION | SAFE    | 0.00       | UNCERTAIN        | UNCERTAIN        |

| Barrier        | Verdict | Confidence | LLM <sub>1</sub> | LLM <sub>2</sub> |
|----------------|---------|------------|------------------|------------------|
| NATURAL_PROOFS | SAFE    | 0.00       | UNCERTAIN        | UNCERTAIN        |
| ALGEBRIZATION  | SAFE    | 0.00       | UNCERTAIN        | UNCERTAIN        |
| KARP_LIPTON    | SAFE    | 0.00       | UNCERTAIN        | UNCERTAIN        |

## 4. Novelty audit

**Verdict:** NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

## 5. Empirical test harness

**Multi-seed protocol:** 5 seeds (11, 23, 37, 53, 71)

**Sandbox:** isolated subprocess, stdlib-only,  $\leq 90$ s wall time per cycle **Execution:** rc=1, elapsed=2.4s

### 5.1 Generated Python source

```
import random
import math

def gcd(a, b):
    while b:
        a, b = b, a % b
    return a

def lcm(a, b):
    return abs(a*b) // gcd(a, b)

def factorial(n):
    if n == 0 or n == 1:
        return 1
    result = 1
    for i in range(2, n + 1):
        result *= i
    return result

def hook_length_formula(shape):
    n = len(shape)
    total = 1
    for i in range(n):
        for j in range(len(shape[i])):
            total //= (i + j + 1)
            total *= shape[i][j] + i + j + 1
    return total

def schur_weyl_decomposition_components(n, m):
    if m != n**2:
        return 0
    components = 0
    for k in range(1, n+1):
        components += hook_length_formula([[n-k+i] * (k-i) + [i] * (n-k) for i in range(k)])
    return components

def monotone_circuit_size_bound(n, C):
    return C * math.log2(n)
```

```

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = 10
    m = n**2
    instances_tested = 30
    conjecture_holds = True
    counterexample = ""

    for _ in range(instances_tested):
        clause_indicator_vectors = [[random.choice([0, 1]) for _ in range(n)] for _ in range(m)]
        tensor_product = [sum(v[i] * w[i] for v, w in zip(vec1, vec2)) for vec1, vec2 in zip(*clause_indicator_vectors)]
        C = schur_weyl_decomposition_components(n, m)
        if C < 2*(n/2) / (n**2):
            conjecture_holds = False
            counterexample = f"Failed at n={n}, C={C}"
            break

    return {
        "metric_name": "Schur-Weyl Decomposition Components",
        "metric_value": C,
        "instances_tested": instances_tested,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]]
    if not seeds:
        primes = [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61, 67, 71, 73, 79, 83, 89, 97]
        seeds = primes[:30]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value)**2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif any(r["counterexample"] for r in results):
        counterexample = next(r["counterexample"] for r in results if r["counterexample"])
        first_failing_seed = next(r["seed"] for r in results if r["counterexample"])
        print(f"RESULT: FALSIFIED counterexample={counterexample} first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE insufficient data")

```

## 6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

## 7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

File "/home/ludo/Scrivania/SEC/research/pvsnp\_sandbox/test\_754b3988.py", line 88, in <module>

result = run\_trial(seed)

^^^^^^^^^^^^^^^^

File "/home/ludo/Scrivania/SEC/research/pvsnp\_sandbox/test\_754b3988.py", line 64, in run\_trial

tensor\_product = [sum(v[i] \* w[i] for v, w in zip(vec1, vec2)) for vec1, vec2 in zip(\*clause\_indicat

^^^^^^^^^^

ValueError: too many values to unpack (expected 2)

## 8. Critic adversarial review

**Critic verdict:** CHALLENGE

**Critic reasoning:**

skipped: test crashed before producing data

## 9. Final verdict & safety rail

**Verdict:** INCONCLUSIVE

**Reasoning:**

Test crashed during tensor product computation, preventing data collection. The ValueError indicates a critical error in the implementation that needs debugging. | next: Fix the unpacking error in the tensor product calculation and revalidate the test setup

## 11. Audit log (LLM calls)

**Total LLM calls:** 9

| # | Phase           | Provider      | Model            | Tokens in | out | Latency (ms) |
|---|-----------------|---------------|------------------|-----------|-----|--------------|
| 1 | propose         | ollama_remote | qwen3:8b         | 0         | 0   | 112040       |
| 2 | propose         | ollama_remote | qwen3:8b         | 0         | 0   | 110169       |
| 3 | propose         | ollama_remote | qwen3:8b         | 0         | 0   | 59438        |
| 4 | preregistration | ollama_remote | qwen3:8b         | 0         | 0   | 25632        |
| 5 | novelty         | ollama_remote | qwen3:8b         | 0         | 0   | 20673        |
| 6 | novelty         | ollama_remote | qwen3:8b         | 0         | 0   | 14932        |
| 7 | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 11815        |
| 8 | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 23205        |
| 9 | judge           | ollama_remote | qwen3:8b         | 0         | 0   | 120546       |

**Totals:** 0 input tokens, 0 output tokens, ~\$0.0000 cost, 498450 ms total latency. Provider mix: {'ollama\_remote': 9}

(full prompt+response transcripts available in research/audit/965ac372534a.jsonl)

## 12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice

3. The full LLM transcripts are in `research/audit/965ac372534a.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

**Sandbox file:** `research/pvsnp_sandbox/test_*.py` (per-cycle)

**Replay tarball:** `research/replay/965ac372534a.tar.gz` (if generated)